12

Clasificadores basados en árboles de decisión

Un **árbol de decisión** es un árbol en cuyos **nodos existen determinadas consultas** acerca de los valores de un atributo determinado.

Los nodos terminales u **hojas** tienen asociada una etiqueta, es decir, un **valor para el atributo target**.

Para clasificar una observación **se empieza por el nodo raíz** y en cada nodo, dependiendo de la respuesta a la consulta asociada al nodo, se avanza a uno de los nodos hijos.

El proceso se repite, y se va descendiendo por el árbol, hasta llegar a un nodo terminal.

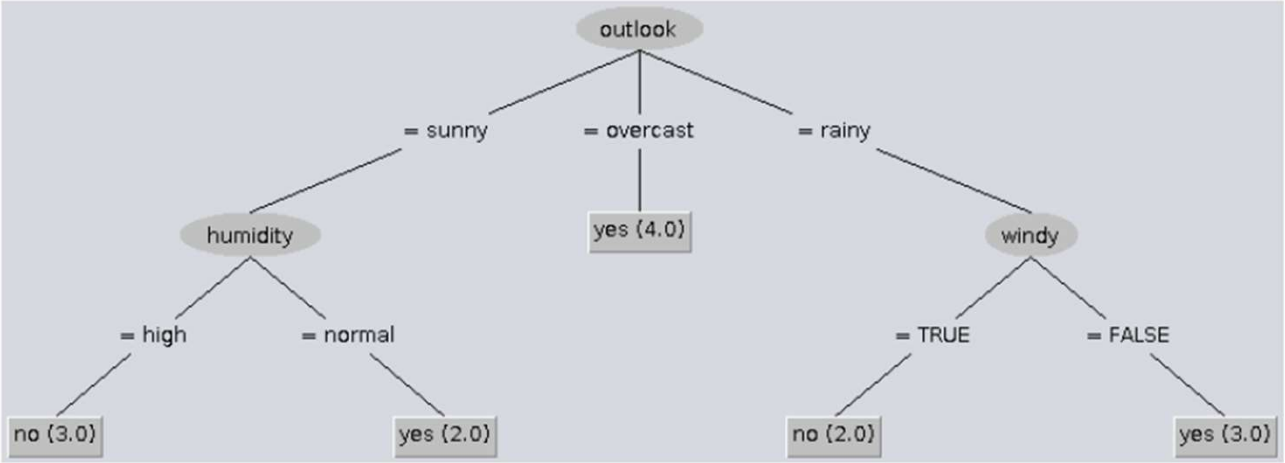
La observación se clasifica entonces **con el valor de la etiqueta** asociado al nodo hoja o **terminal**.

El siguiente **ejemplo** es un clasificador para **decidir si un grupo de amigos juegan su habitual partido de baloncesto** del sábado por la mañana.

A partir del dataset del que se dispone se ha inducido un árbol de decisión.

La etiqueta o atributo target es “play”, con valores “yes” y “no”.

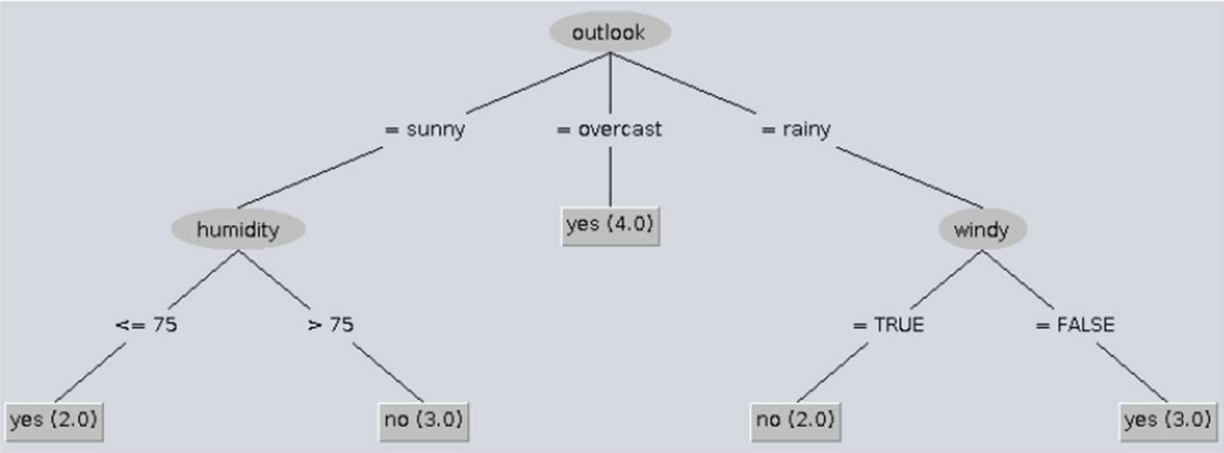
	outlook	temperature	humidity	windy	play
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no



También se pueden utilizar **árboles de decisión para datos numéricos**.

En este caso, las consultas en los nodos pueden conducir a distintos **nodos según intervalos de valores**, y no solo valores individuales, como en el ejemplo anterior, en el que solo habían atributos categóricos.

	outlook	temperature	humidity	windy	play
1	sunny	85.0	85.0	FALSE	no
2	sunny	80.0	90.0	TRUE	no
3	overcast	83.0	86.0	FALSE	yes
4	rainy	70.0	96.0	FALSE	yes
5	rainy	68.0	80.0	FALSE	yes
6	rainy	65.0	70.0	TRUE	no
7	overcast	64.0	65.0	TRUE	yes
8	sunny	72.0	95.0	FALSE	no
9	sunny	69.0	70.0	FALSE	yes
10	rainy	75.0	80.0	FALSE	yes
11	sunny	75.0	70.0	TRUE	yes
12	overcast	72.0	90.0	TRUE	yes
13	overcast	81.0	75.0	FALSE	yes
14	rainy	71.0	91.0	TRUE	no



VENTAJAS DE LOS ÁRBOLES DE DECISIÓN

- **No es necesario ningún preproceso** ni normalización de los datos.
- Se pueden utilizar con **atributos categóricos y numéricos**.
- **Interpretabilidad**. Son completamente transparentes. Con un árbol de decisión queda muy patente la relevancia de cada uno de los atributos para la clasificación y cómo y en qué contexto se tiene en cuenta cada uno.
- **Facilidad para implementarlos** en cualquier entorno de programación.
- **Flexibilidad**. Se pueden establecer criterios de parada, y se pueden realizar podas al árbol, una vez generado.

INCONVENIENTES DE LOS ÁRBOLES DE DECISIÓN

- Los **atributos continuos contribuyen** a que se creen **árboles más grandes** y complejos y, por tanto, **menos interpretables**.

- **Tendencia al sobreaprendizaje**. Por su propia flexibilidad y adaptabilidad. Este problema es especialmente importante cuantos más atributos y menos observaciones se tengan. También se agrava especialmente con datasets desequilibrados (imbalanced).

- **Pequeños cambios** en los datos de aprendizaje p**ueden provocar cambios drásticos en los árboles** inducidos mediante aprendizaje.

Clasificadores basados en árboles de decisión en Scikit-Learn

La clase **DecisionTreeClassifier** implementa árboles de decisión en Scikit-Learn.

Esta clase **implementa árboles CART** (Classification and Regression Trees).

El algoritmo CART **soporta variables target numéricas** (es decir, que se puede usar para regresión, como se verá en la siguiente unidad, dedicada a regresión).

Según la documentación de DecisionTreeClassifier, su implementación de CART **no soporta atributos predictores categóricos** por ahora.
https://scikit-learn.org/stable/modules/tree.html#tree-algorithms-id3-c4-5-c5-0-and-cart

"scikit-learn usesan optimized version of the CART algorithm; however, the scikit-learn implementation does not support categorical variables for now".

Pero da buenos resultados si se convierten sus valores a numéricos con LabelEncoder.
Eso sí, los árboles salen más complejos que los que se obtienen con un algoritmo para árboles de decisión que funcione con atributos predictores categóricos.

Por tanto, debemos tener en cuenta que:

- Podemos **evitar la tendencia al sobreajuste** de los árboles a partir de una **buena selección de hiperparámetros.**

- El problema del sobreajuste **se acentúa en datasets desbalanceados** y, por tanto, debemos **balancear previamente el dataset** si fuese necesario.

- Aunque los árboles de decisión pueden utilizar todo tipo de atributos predictores, por ahora la clase **DecisionTreeClassifier solo soporta atributos numéricos** por tanto **debemos usar LabelEncoder** sobre los atributos categóricos.

Además:

- DecisionTreeClassifier **tiene soporte para valores nulos** cuando splitter='best' y criterion es 'gini', 'entropy', o 'log_loss'. (El valor por defecto para splitter es 'best', y para criterion es 'gini'.)

- Por tanto, a menos que se dé algún otro valor a cualquiera de estos parámetros **no es necesario, y será mejor, no eliminar los valores nulos**.

Clase DecisionTreeClassifier de Scikit-Learn

PARÁMETROS DEL CONSTRUCTOR	criterion	Función para medir la calidad de una partición de un nodo. En particular, la variabilidad media del valor del atributo target en los distintos nodos hijos. Los posibles valores son: • "gini". Índice de Gini. Es el valor por defecto. • "entropy", "log_loss". Entropía. La entropía suele dar resultados ligeramente mejores, pero tiene mayor coste computacional.
	splitter	Estrategia para elegir la partición de los nodos. Los posibles valores son: • "best". La mejor partición. Es el valor por defecto. • "random". La mejor partición al azar. Es más rápido, pero puede dar peores resultados.
	max_depth	Máxima profundidad del árbol. El valor por defecto es None. Con este valor, los nodos se expanden hasta que solo contienen observaciones de la misma clase o mientras no contengan menos de min_samples_split observaciones.
	min_samples_split	Mínimo números de observaciones para partir un nodo. El valor por defecto es 2. Si es un int, indica el número mínimo de nodos observaciones. Si es un float, indica la proporción del número total de observaciones.

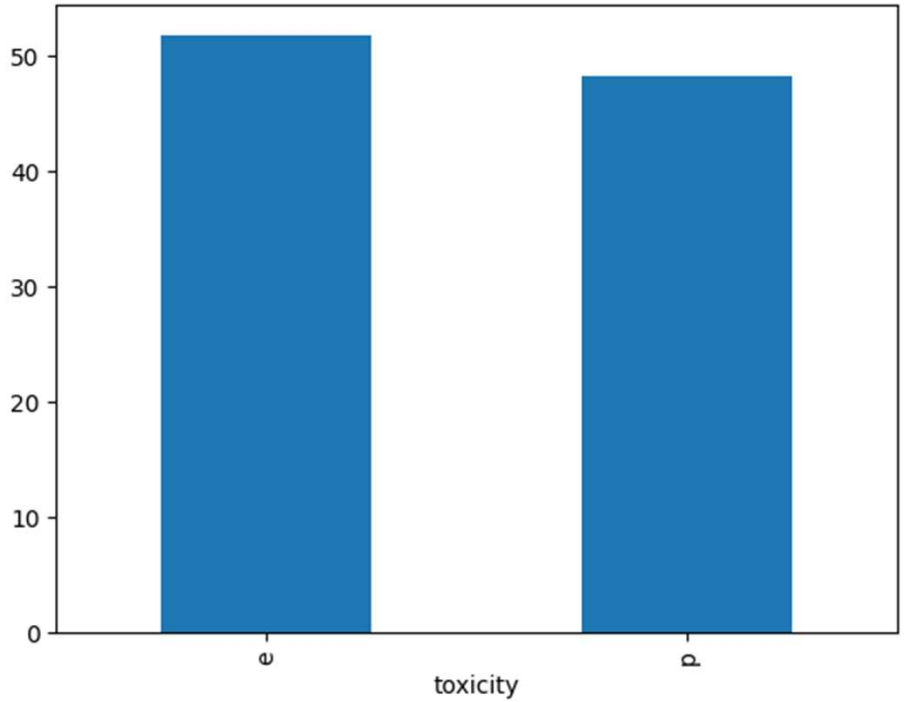
PARÁMETROS DEL CONSTRUCTOR	min_samples_leaf	**Mínimo número de observaciones que debe tener un nodo hoja. Es decir, un nodo solo se partirá si en cada nuevo nodo quedan al menos este número de observaciones. El valor por defecto es 1. Y puede ser un int que indica el número mínimo de observaciones o un float que indica la proporción del número total de observaciones.**
	max_features	Número de atributos predictores (features) que hay que considerar para la mejor partición. El valor por defecto es None. • None. Considerar todos los atributos predictores. • Si es un int, considerar ese número de atributos predictores. • Si es un float, considerar esa proporción del número de atributos predictores. • "auto" o "sqrt". Considerar la raíz cuadrada del número de atributos predictores. • "log2". Considerar el logaritmo en base 2 del número de atributos predictores.
	ccp_alpha	Máxima complejidad para el algoritmo de poda de mínimo coste-complejidad. Se elegirá para podar el subárbol con la mayor complejidad que sea menor que el valor de este parámetro. El valor por defecto es 0, que significa que no se hará poda.

MÉTODOS	get_depth()	**Devuelve la profundidad del árbol**
	get_n_leaves()	Devuelve el número de hojas del árbol
ATRIBUTOS	feature_importances_	Devuelve la importancia de los atributos predictores
	tree__	Devuelve el propio árbol, se puede usar para su representación gráfica

Ejemplo 4 Setas_DecisionTree

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Carga de datos
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-
databases/mushroom/agaricus-lepiota.data',names=['toxicity', 'cap-
shape', 'cap-surface', 'cap-color', 'bruises', 'odor', 'gill-
attachment', 'gill-spacing', 'gill-size','gill-color', 'stalk-
shape', 'stalk-root', 'stalk-surface-above-ring', 'stalk-surface-
below-ring', 'stalk-color-above-ring','stalk-color-below-ring',
'veil-type', 'veil-color', 'ring-number', 'ring-type', 'spore-print-
color', 'population', 'habitat'],na_values = '?'
)
df.info()
```

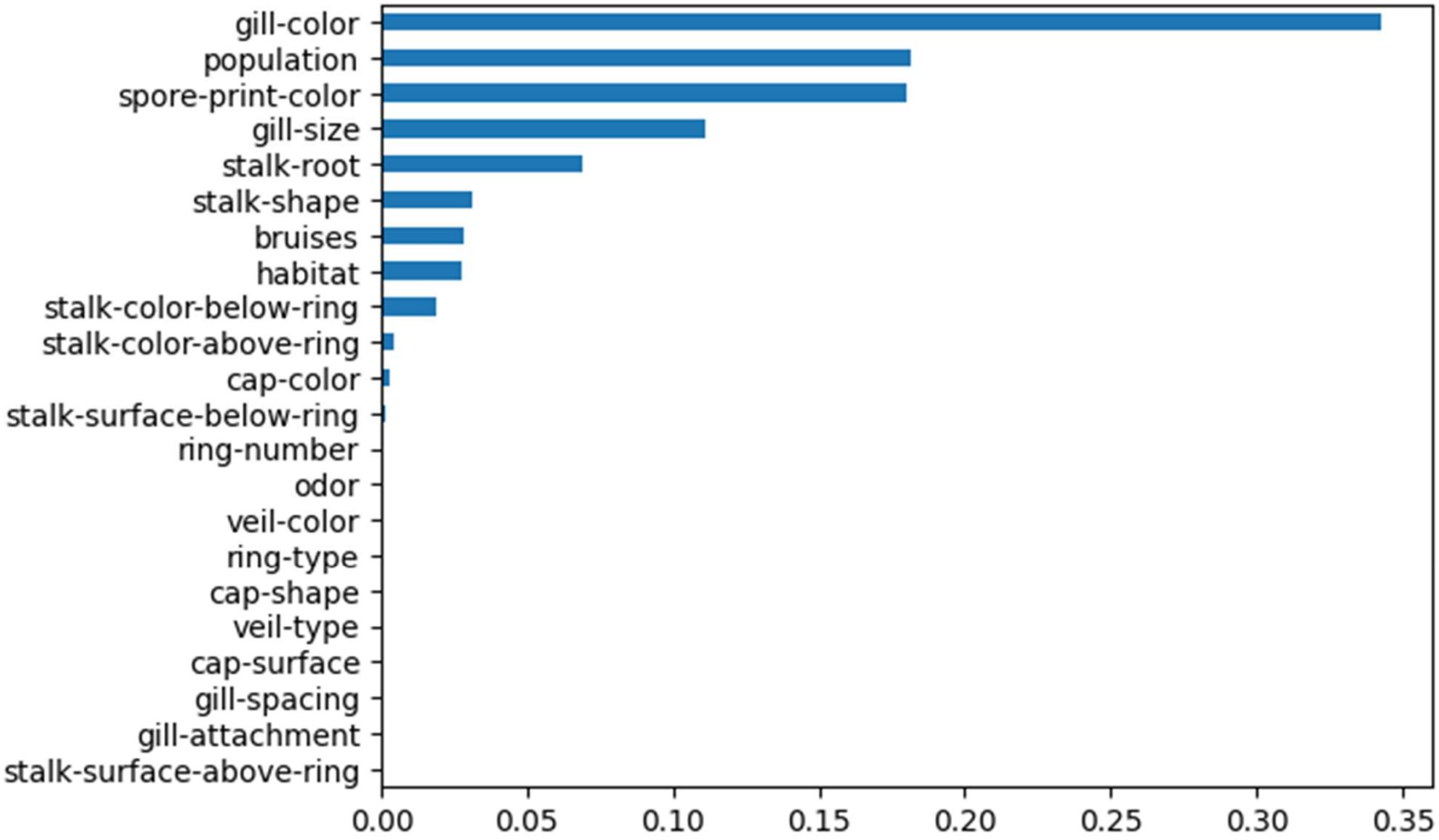
```
# Variable target: toxicity
# Valores:
#     'e': comestible (edible)
#     'p': venenosa (poisonous)

target = 'toxicity'
```



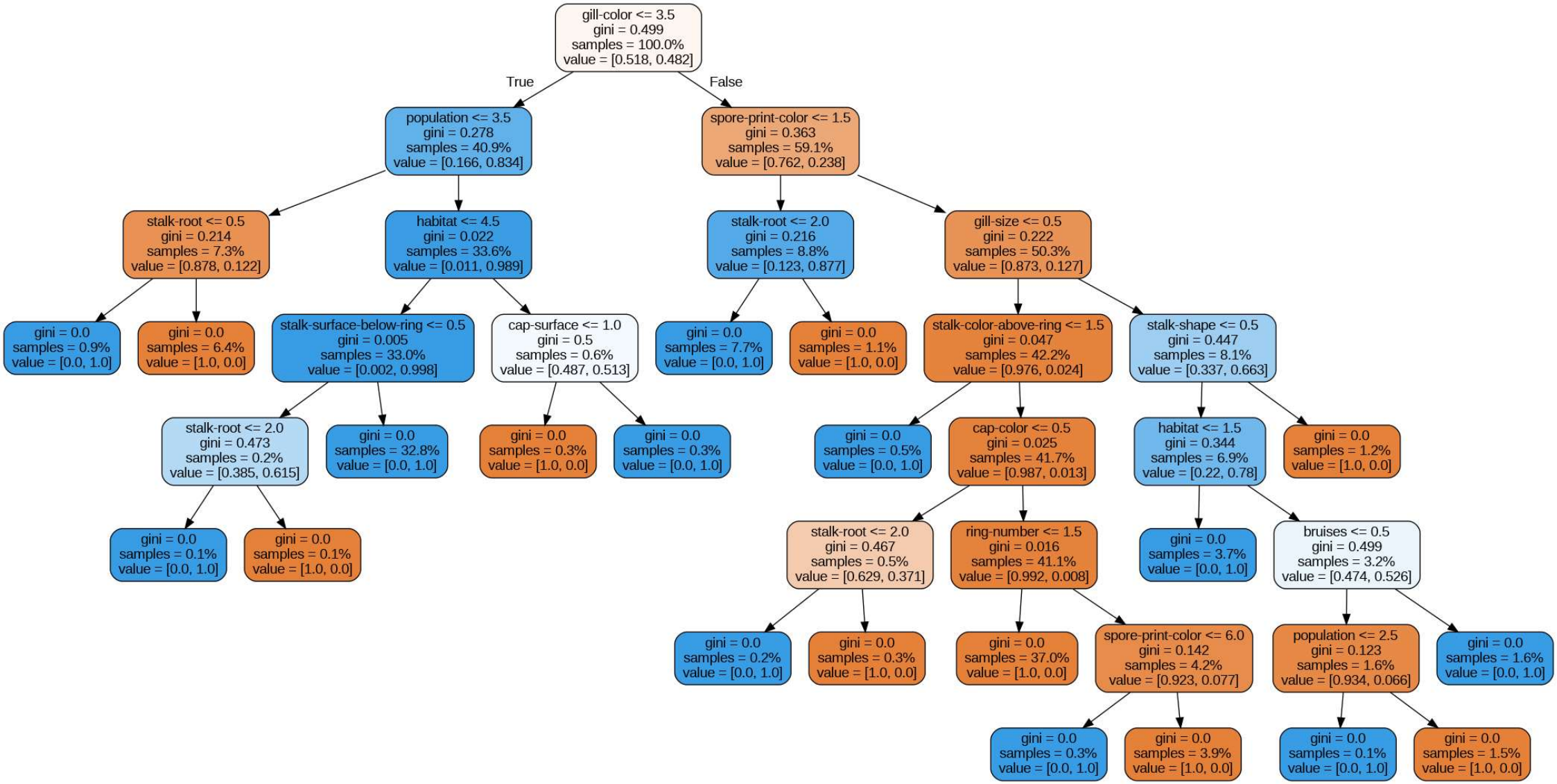

```
# Visualizamos en un gráfico el número de observaciones de cada
# clase para saber si el dataset está desbalanceado

(100*df[target].value_counts()/df.shape[0]).sort_index().plot(k
ind='bar')
```

```python
# Recodificar como números todos los atributos categóricos
for atrib in df.columns:
  if atrib != target:
    df[atrib] = LabelEncoder().fit_transform(df[atrib].values)


x = df.drop([target], axis=1)
y = df[target]

RANDOM_SEED = 134

x_train, x_test, y_train, y_test = train_test_split(x, y,
train_size=0.8, random_state=RANDOM_SEED)

clf = DecisionTreeClassifier()
clf.fit(x_train, y_train)
```

```python
# Visualizamos el accuracy tanto en entrenamiento como
# validación
acc_rate_train = accuracy_score(y_test, clf.predict(x_test))
print("Tasa de acierto en dataset de validación: ", acc_rate_train)

acc_rate_valid = accuracy_score(y_train, clf.predict(x_train))
print("Tasa de acierto en dataset de entrenamiento: ",
acc_rate_valid)

# Vemos un array con las clases que ha aprendido nuestro modelo
clf.classes_

# Creamos un array de pandas con el nombre de la clase como índice y
# su importancia como valor y lo graficamos en barras
pd.Series(clf.feature_importances_,
index=clf.feature_names_in_).nlargest(clf.feature_names_in_.shape[0]
).sort_values(ascending=True).plot(kind='barh')
```


gill-color
population
spore-print-color
gill-size
stalk-root
stalk-shape
bruises
habitat
stalk-color-below-ring
stalk-color-above-ring
cap-color
stalk-surface-below-ring
ring-number
odor
veil-color
ring-type
cap-shape
veil-type
cap-surface
gill-spacing
gill-attachment
stalk-surface-above-ring
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35

```python
# Visualizamos el árbol de decisión obtenido

import pydot
from IPython.display import Image
from sklearn import tree
from io import StringIO

dot_data = StringIO()

tree.export_graphviz(clf, out_file=dot_data,
proportion=True,feature_names = x_train.columns,rounded = True,
filled = True)

graph = pydot.graph_from_dot_data(dot_data.getvalue())

Image(graph[0].create_png())
```


gill-color <= 3.5
gini = 0.499
samples = 100.0%
value = [0.518, 0.482]
True
False
population <= 3.5
gini = 0.278
samples = 40.9%
value = [0.166, 0.834]
spore-print-color <= 1.5
gini = 0.363
samples = 59.1%
value = [0.762, 0.238]
stalk-root <= 0.5
gini = 0.214
samples = 7.3%
value = [0.878, 0.122]
habitat <= 4.5
gini = 0.022
samples = 33.6%
value = [0.011, 0.989]
stalk-root <= 2.0
gini = 0.216
samples = 8.8%
value = [0.123, 0.877]
gill-size <= 0.5
gini = 0.222
samples = 50.3%
value = [0.873, 0.127]
gini = 0.0
samples = 0.9%
value = [0.0, 1.0]
gini = 0.0
samples = 6.4%
value = [1.0, 0.0]
stalk-surface-below-ring <= 0.5
gini = 0.005
samples = 33.0%
value = [0.002, 0.998]
cap-surface <= 1.0
gini = 0.5
samples = 0.6%
value = [0.487, 0.513]
gini = 0.0
samples = 7.7%
value = [0.0, 1.0]
gini = 0.0
samples = 1.1%
value = [1.0, 0.0]
stalk-color-above-ring <= 1.5
gini = 0.047
samples = 42.2%
value = [0.976, 0.024]
stalk-shape <= 0.5
gini = 0.447
samples = 8.1%
value = [0.337, 0.663]
stalk-root <= 2.0
gini = 0.473
samples = 0.2%
value = [0.385, 0.615]
gini = 0.0
samples = 32.8%
value = [0.0, 1.0]
gini = 0.0
samples = 0.3%
value = [1.0, 0.0]
gini = 0.0
samples = 0.3%
value = [0.0, 1.0]
gini = 0.0
samples = 0.5%
value = [0.0, 1.0]
cap-color <= 0.5
gini = 0.025
samples = 41.7%
value = [0.987, 0.013]
habitat <= 1.5
gini = 0.344
samples = 6.9%
value = [0.22, 0.78]
gini = 0.0
samples = 1.2%
value = [1.0, 0.0]
gini = 0.0
samples = 0.1%
value = [0.0, 1.0]
gini = 0.0
samples = 0.1%
value = [1.0, 0.0]
stalk-root <= 2.0
gini = 0.467
samples = 0.5%
value = [0.629, 0.371]
ring-number <= 1.5
gini = 0.016
samples = 41.1%
value = [0.992, 0.008]
gini = 0.0
samples = 3.7%
value = [0.0, 1.0]
bruises <= 0.5
gini = 0.499
samples = 3.2%
value = [0.474, 0.526]
gini = 0.0
samples = 0.2%
value = [0.0, 1.0]
gini = 0.0
samples = 0.3%
value = [1.0, 0.0]
gini = 0.0
samples = 37.0%
value = [1.0, 0.0]
spore-print-color <= 6.0
gini = 0.142
samples = 4.2%
value = [0.923, 0.077]
population <= 2.5
gini = 0.123
samples = 1.6%
value = [0.934, 0.066]
gini = 0.0
samples = 1.6%
value = [0.0, 1.0]
gini = 0.0
samples = 0.3%
value = [0.0, 1.0]
gini = 0.0
samples = 3.9%
value = [1.0, 0.0]
gini = 0.0
samples = 0.1%
value = [0.0, 1.0]
gini = 0.0
samples = 1.5%
value = [1.0, 0.0]

```
# Podemos asignar un color que dependa de la clase con el siguiente codigo
import pydotplus

dot_data = tree.export_graphviz(clf,
                                feature_names=df.columns[1:],
                                out_file=None,
                                filled=True,
                                rounded=True,
                                special_characters=True)
graph = pydotplus.graph_from_dot_data(dot_data)
nodes = graph.get_node_list()
#colors = []
for node in nodes:
    if node.get_label():
        values = [int(float(ii)) for ii in node.get_label().split('value =
[')[1].split(']')[0].split(',')]
        values = [int(255 * v / sum(values)) for v in values]
        color = '#{:02x}{:02x}00'.format(values[1], values[0])
#         colors.append(color)
        node.set_fillcolor(color)

Image(graph.create_png())
```

gill-color ≤ 3.5
gini = 0.499
samples = 6499
value = [3366, 3133]
True
False
population ≤ 3.5
gini = 0.278
samples = 2661
value = [443, 2218]
spore-print-color ≤ 1.5
gini = 0.363
samples = 3838
value = [2923, 915]
stalk-root ≤ 0.5
gini = 0.214
samples = 477
value = [419, 58]
habitat ≤ 4.5
gini = 0.022
samples = 2184
value = [24.0, 2160.0]
stalk-root ≤ 2.0
gini = 0.216
samples = 569
value = [70, 499]
gill-size ≤ 0.5
gini = 0.222
samples = 3269
value = [2853.0, 416.0]
gini = 0.0
samples = 58
value = [0, 58]
gini = 0.0
samples = 419
value = [419, 0]
stalk-surface-below-ring ≤ 0.5
gini = 0.005
samples = 2145
value = [5, 2140]
cap-surface ≤ 1.0
gini = 0.5
samples = 39
value = [19, 20]
gini = 0.0
samples = 499
value = [0, 499]
gini = 0.0
samples = 70
value = [70, 0]
stalk-color-above-ring ≤ 1.5
gini = 0.047
samples = 2741
value = [2675, 66]
stalk-shape ≤ 0.5
gini = 0.447
samples = 528
value = [178, 350]
stalk-root ≤ 2.0
gini = 0.473
samples = 13
value = [5, 8]
gini = 0.0
samples = 2132
value = [0, 2132]
gini = 0.0
samples = 19
value = [19, 0]
gini = 0.0
samples = 20
value = [0, 20]
gini = 0.0
samples = 32
value = [0, 32]
cap-color ≤ 0.5
gini = 0.025
samples = 2709
value = [2675, 34]
habitat ≤ 1.5
gini = 0.344
samples = 449
value = [99, 350]
gini = 0.0
samples = 79
value = [79, 0]
gini = 0.0
samples = 8
value = [0, 8]
gini = 0.0
samples = 5
value = [5, 0]
stalk-root ≤ 2.0
gini = 0.467
samples = 35
value = [22, 13]
ring-number ≤ 1.5
gini = 0.016
samples = 2674
value = [2653, 21]
gini = 0.0
samples = 240
value = [0, 240]
bruises ≤ 0.5
gini = 0.499
samples = 209
value = [99, 110]
gini = 0.0
samples = 13
value = [0, 13]
gini = 0.0
samples = 22
value = [22, 0]
gini = 0.0
samples = 2402
value = [2402, 0]
spore-print-color ≤ 6.0
gini = 0.142
samples = 272
value = [251, 21]
population ≤ 2.5
gini = 0.123
samples = 106
value = [99, 7]
gini = 0.0
samples = 103
value = [0, 103]
gini = 0.0
samples = 21
value = [0, 21]
gini = 0.0
samples = 251
value = [251, 0]
gini = 0.0
samples = 7
value = [0, 7]
gini = 0.0
samples = 99
value = [99, 0]